LAPORAN PRAKTIKUM
TEKNOLOGI IOT II

Modul 6 Tugas

Dosen Pengampu:

1. Dr. Ir. Septafiansyah Dwi Putra - NIP : 198809112009121001
2. Agus Ambarwari, S.Pd., M.Kom. - NIP : 199208082019031020



Disusun oleh :

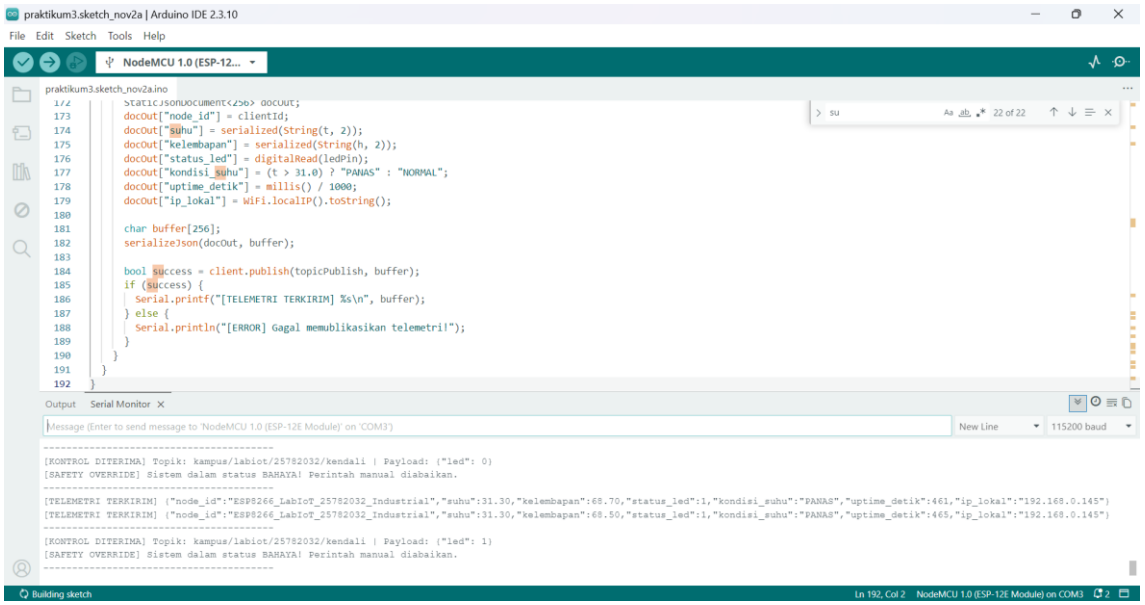
Ananda Haris Rahmadi (25782032)

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026



HIVEMQ CLOUD
Need a fully managed MQTT broker?
Get your own Cloud broker and connect up to 100 devices for free.
Get your free account

Connection

connected

Host: broker.hivemq.com
Port: 8884
ClientID: clientId-qB0ndCNpTm
Disconnect
Username
Password
Keep Alive: 60
SSL
Clean Session
Last-Will Topic
Last-Will QoS: 0
Last-Will Retain
Last-Will Messsage

Publish

Topic: kampus/labiot/25782032/kendali
QoS: 1
Retain
Publish
Message: {"led": 1}

Subscriptions

Add New Topic Subscription

Qos: 2
kampus/labiot/25782...

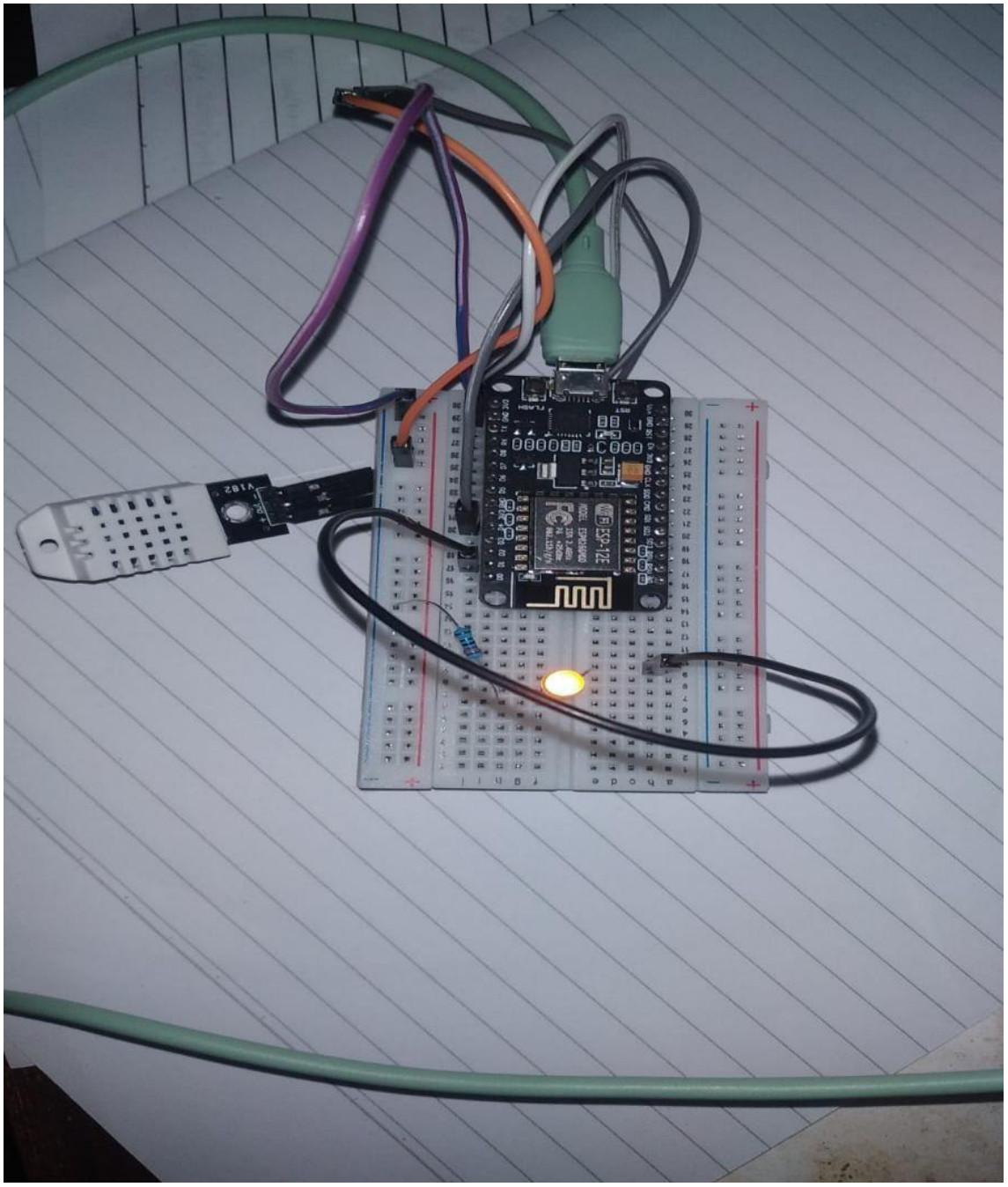
Messages

2026-11-02 15:51:46 Topic: kampus/labiot/25782032/kendali Qos: 1
{"led": 1}

2026-11-02 15:51:39 Topic: kampus/labiot/25782032/kendali Qos: 1
{"led": 0}

2026-11-02 15:50:21 Topic: kampus/labiot/25782032/kendali Qos: 1
{"led": 1}

2026-11-02 15:50:19 Topic: kampus/labiot/25782032/kendali Qos: 1
{"led": 1}



Kode:

```
#include <ESP8266WiFi.h>

#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi
const char* ssid = "Network_eng";
const char* password = "d585de2865";
```

```
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782032_Industrial";

// Definisi Topik MQTT
const char* topicPublish   = "kampus/labiot/25782032/telemetri";
const char* topicSubscribe = "kampus/labiot/25782032/kendali";
const char* topicAlert     = "kampus/labiot/25782032/alert";
const char* topicStatus    = "kampus/labiot/25782032/status";

// Alokasi Pin Perangkat Keras
const byte dhtPin = 2; // Pin D4 (GPIO 2)
const byte ledPin = 5; // Pin D1 (GPIO 5)
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long lastTelemetryTime = 0;
const unsigned long telemetryInterval = 4000; // Siklus telemetri 4 detik

// State Latch (Flag) untuk mencegah spamming pesan alert bahaya
bool isOverheatState = false;

// Callback untuk menerima perintah kendali manual dari operator luar
void callback(char* topic, byte* payload, unsigned int length) {
  char incomingBuffer[length + 1];
  memcpy(incomingBuffer, payload, length);
  incomingBuffer[length] = '\0';

  Serial.println("----------------------------------------");
  Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n", topic,
incomingBuffer);

  // Jika kondisi sedang BAHAYA (overheat), abaikan perintah manual
override dari luar demi keamanan
  float currentTemp = dht.readTemperature();
  if (!isnan(currentTemp) && currentTemp > 31.0) {
    Serial.println("[SAFETY OVERRIDE] Sistem dalam status BAHAYA!
Perintah manual diabaikan.");
    Serial.println("----------------------------------------");
    return;
  }
```

```
  StaticJsonDocument<256> doc;
  DeserializationError err = deserializeJson(doc, incomingBuffer);
  if (err) {
    Serial.println("Gagal membedah payload JSON!");
    return;
  }

  // Kendali manual LED saat kondisi normal
  if (doc.containsKey("led")) {
    int val = doc["led"];
    digitalWrite(ledPin, val == 1 ? HIGH : LOW);
    Serial.printf("--> [MANUAL OVERRIDE] Status LED diubah: %s\n", val ==
1 ? "MENYALA" : "PADAM");
  }
  Serial.println("----------------------------------------");
}

// Fungsi Reconnect dengan pendaftaran Last Will and Testament (LWT)
void reconnect() {
  while (!client.connected()) {
    Serial.print("Menghubungkan kembali ke Broker MQTT...");

    // Payload pesan wasiat (LWT)
    const char* lwtMessage = "{\"status\": \"OFFLINE\", \"keterangan\":
\"Perangkat Terputus Tak Terduga\"}";

    // Koneksi dengan parameter LWT: (clientId, willTopic, willQos,
willRetain, willMessage)
    if (client.connect(clientId, topicStatus, 1, true, lwtMessage)) {
      Serial.println(" Sukses Terhubung!");

      // Subskripsi kembali ke topik kendali
      client.subscribe(topicSubscribe);
      Serial.printf("Subskripsi Topik Kendali: %s\n", topicSubscribe);

      // Publikasikan status ONLINE sesaat setelah berhasil terhubung
      StaticJsonDocument<128> statusDoc;
      statusDoc["status"] = "ONLINE";
      statusDoc["keterangan"] = "Perangkat Beroperasi Normal";
      char statusBuffer[128];
      serializeJson(statusDoc, statusBuffer);

      client.publish(topicStatus, statusBuffer, true); // Published
dengan retain true
      Serial.printf("[STATUS] Publikasi Online ke %s\n", topicStatus);

    } else {
```

```
      Serial.printf("Gagal terhubung (rc=%d). Coba lagi dalam 4
detik...\n", client.state());
      delay(4000);
    }
  }
}

void setup() {
  Serial.begin(115200);
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  dht.begin();

  // Koneksi Jaringan Wi-Fi
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Menghubungkan Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWi-Fi Terkoneksi!");
  Serial.print("IP Address lokal: ");
  Serial.println(WiFi.localIP());

  client.setServer(mqttServer, mqttPort);
  client.setCallback(callback);
}

void loop() {
  if (!client.connected()) {
    reconnect();
  }
  client.loop(); // Memelihara sesi TCP dan memproses callback masuk

  unsigned long now = millis();
  if (now - lastTelemetryTime >= telemetryInterval) {
    lastTelemetryTime = now;

    float t = dht.readTemperature();
    float h = dht.readHumidity();

    if (!isnan(t) && !isnan(h)) {
      // 1. RULE ENGINE & SAFETY LOGIC (Edge Safety Engine)
      if (t > 31.0) {
        // Kondisi Suhu Panas / Bahaya Overheat
```

```
        digitalWrite(ledPin, HIGH); // Otomatis nyalakan LED indikator
bahaya

        // Kirim alert sekali saja saat transisi masuk ke kondisi bahaya
(mencegah spamming tiap milidetik)
        if (!isOverheatState) {
          isOverheatState = true;

          StaticJsonDocument<256> alertDoc;
          alertDoc["status"] = "BAHAYA";
          alertDoc["parameter"] = "OVERHEAT";
          alertDoc["nilai_suhu"] = serialized(String(t, 2));

          char alertBuffer[256];
          serializeJson(alertDoc, alertBuffer);

          bool alertSuccess = client.publish(topicAlert, alertBuffer);
          if (alertSuccess) {
            Serial.printf("[ALERT BAHAYA TERKIRIM] %s\n", alertBuffer);
          }
        }
      } else {
        // Kondisi Normal (Suhu <= 31.0°C)
        if (isOverheatState) {
          // Reset state latch saat suhu kembali normal
          isOverheatState = false;
          digitalWrite(ledPin, LOW); // Matikan LED peringatan
          Serial.println("[INFO] Suhu kembali normal. Sistem pulih dari
status BAHAYA.");
        }
      }

      // 2. SERIALISASI TELEMETRI RUTIN DENGAN KLASIFIKASI LINGKUNGAN
      StaticJsonDocument<256> docOut;
      docOut["node_id"] = clientId;
      docOut["suhu"] = serialized(String(t, 2));
      docOut["kelembapan"] = serialized(String(h, 2));
      docOut["status_led"] = digitalRead(ledPin);
      docOut["kondisi_suhu"] = (t > 31.0) ? "PANAS" : "NORMAL";
      docOut["uptime_detik"] = millis() / 1000;
      docOut["ip_lokal"] = WiFi.localIP().toString();

      char buffer[256];
      serializeJson(docOut, buffer);

      bool success = client.publish(topicPublish, buffer);
      if (success) {
```

```
        Serial.printf("[TELEMETRI TERKIRIM] %s\n", buffer);
      } else {
        Serial.println("[ERROR] Gagal memublikasikan telemetri!");
      }
    }
  }
}
```